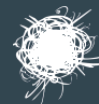


BTRFS для самых маленьких / Хабр

 habr.com/ru/companies/veeam/articles/458250

July 24, 2019

Хабр



BTRFS для самых маленьких

Veeam Software

BTRFS для самых маленьких

Доброго времени суток, Хабравчане. Я работаю в компании Veeam Software и являюсь одним из разработчиков нашего решения для бэкапа линукс систем. По роду занятий мне довелось столкнуться с BTRFS. Совсем недавно она перешла из статуса «еще не пригодно» в статус «стабильна». И пока её первые пользователи в сети обсуждали проблемные места и вопросы стабильности, мы в Veeam тыкали её палочкой и пытались бэкапить. Получалось, мягко говоря, не очень — слишком уж она другая, не похожая на традиционные файловые системы. Пришлось изучить немало аспектов и собрать множество граблей, прежде чем научились с ней работать. В процессе изучения BTRFS сумела произвести на меня впечатление как в хорошем смысле, так и не очень. Уверен, она не оставит равнодушным ни одного айтишника из мира линукс: одни будут плевать, другие восхвалять.

Если вы слышали об этой файловой системе, но не знаете, зачем она, интересуетесь подробностями или ищете, с чего начать знакомство с ней — приглашаю под кат.

Введение

BTRFS (B-Tree Filesystem) — файловая система для Unix-подобных операционных систем, основанная на технике «Copy on Write» (CoW), призванная обеспечить легкость масштабирования файловой системы, высокую степень надежности и сохранности данных, гибкость настроек и легкость администрирования, сохраняя при этом высокую скорость работы. По крайней мере, так гласит [главная вики-страничка](#).

Для соблюдения формальностей перечислим основные возможности btrfs:

- Максимальный размер файла 2^{64} байт
- Динамическая таблица inode
- Дедупликация данных
- Эффективное хранение файлов как очень малых, так и очень больших размеров
- Создание сабволюмов и снапшотов
- Квоты на размеры сабволюмов
- Контрольные суммы для данных и метаданных
- Возможность объединить несколько накопителей в единую файловую систему
- Создание RAID конфигурации на уровне файловой системы
- Сжатие данных
- Дефрагментация данных на лету

Сразу хочу предупредить о том, что BTRFS активно развивается, и некоторые моменты могут отличаться от версии к версии. По ссылке — <https://btrfs.wiki.kernel.org/index.php/Changelog> можно узнать, когда какой функционал был добавлен, изменен или исправлен.

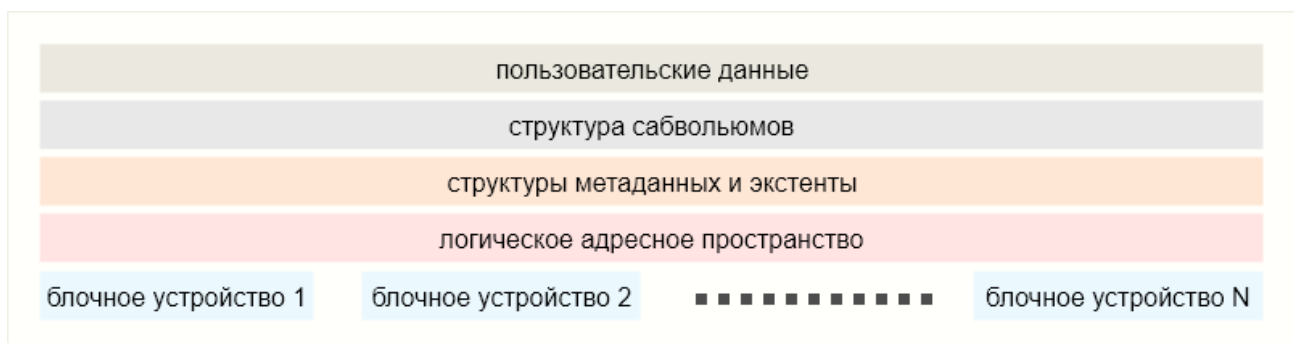
Да, BTRFS — это молодая и современная файловая система, решающая широкий спектр задач, однако не без минусов:

- Активное её развитие приводит к изменению каких-либо ключевых моментов, на которые могут опираться сторонние утилиты при работе с ней.
- Несмотря на заверения разработчиков о стабильности BTRFS, пользователи регулярно сталкиваются с проблемами, потенциально приводящими к потере данных. Как правило, они носят «плавающий» характер, вследствие чего до сих пор не изучены и не исправлены.
- Высокая подверженность фрагментации.
- Скучная и местами устаревшая документация.

Проблемам файловой системы на разных версиях ядер посвящена целая страница — <https://btrfs.wiki.kernel.org/index.php/Gotchas>. Очень советую туда заглядывать — выясняется много интересного и неочевидного.

Структура BTRFS

Упрощенно устройство BTRFS можно разбить на следующие уровни:



На самом низком уровне располагаются блочные устройства, представляя собой одно или несколько отдельных физических адресных пространств (таких же «физических», какими являются сами блочные устройства, но это уже детали). Посредством специальных структур аллоцированные блоки физической памяти объединяются в единое виртуальное адресное пространство.

Структуры метаданных и блоки с пользовательскими данными (экстенты) адресуются уже на логическом уровне. В результате данные, расположенные последовательно на логическом уровне, физически могут находиться на разных блочных устройствах.

Структуры метаданных можно условно разбить на уровни. Классифицировать я их, конечно, не буду — их много, и подобные низкоуровневые подробности — тема отдельной статьи. Здесь важно, что одни структуры в иерархии окажутся более высокоуровневыми, чем другие, а на самом верху окажется структура, представляющая собой сабвольтюм.

Сабволюмы — это своеобразные точки входа, или, правильнее, корневые элементы файловой системы. Они образуют отдельный слой представления данных, который

инкапсулирует работу низших слоев, представляя пользовательские данные уже в привычном нам виде: директории и файлы. Кроме того, сабвольюмы являются ключевым элементом механизма CoW на BTRFS. Одинаковые файлы в двух сабвольюмах могут оказаться одним и тем же набором данных на нижележащих уровнях.

Последний слой — слой данных. Таких, какими видит их пользователь. Это файлы и директории, располагающиеся в сабвольюмах.

Но довольно теории. Пора переходить к практике!

Btrfs-progs

Это штатный комплект утилит для управления BTRFS. В зависимости от дистрибутива пакет с этими утилитами в репозитории может носить различные названия: **btrfsprogs**, **btrfs-progs**, **btrfs-tools** и т.п. Если же в вашем репозитории ничего похожего не оказалось — всегда можно собрать вручную, исходники лежат недалеко — <https://github.com/kdave/btrfs-progs>.

Самые главные утилиты в данном пакете — это **btrfs** и **mkfs.btrfs**. Со второй, думаю, все предельно ясно — она необходима, чтобы создать BTRFS на блочном устройстве. Первая же, **btrfs** — это основная утилита, позволяющая делать все остальное. Этаким «швейцарский нож».

В этой статье я использовал версию v4.15.1. Утилита развивается очень активно, и от версии к версии бывают ощутимые различия. Так что если у вас не оказалось нужной команды, проверьте версию утилиты **btrfs**, возможно, она уже устарела.

Также, скорее всего, в пакете обнаружатся утилиты **btrfsck** и **btrfstune**.

- Первая из них служит для проверки файловой системы на ошибки и для последующих их исправлений, однако, использовать я её не рекомендую — она в статусе *deprecated*, её функционал перемещен в команду **btrfs check**.
- Вторая — позволяет производить некоторые полезные операции над btrfs, например, менять уникальный идентификатор файловой системы (FS UUID), либо включить определенный функционал файловой системы.

Кроме перечисленных выше утилит, в пакете найдутся еще несколько утилит, но они уже в основном нужны для отладки btrfs и в данной статье нам не пригодятся.

Форматирование диска в BTRFS

На практике все проще. Начнем с одного диска.

Форматирование одного диска в btrfs происходит привычной командой:

```
mkfs.btrfs /dev/sdc -L single_drive
```

В ответ утилита выведет в консоль параметры созданной файловой системы:

```
btrfs-progs v4.15.1
See http://btrfs.wiki.kernel.org for more information.

Label:                single_drive
UUID:                 59307d69-6d2f-4d2e-aae2-a5189ad3c256
Node size:            16384
Sector size:          4096
Filesystem size:      1.00GiB
Block group profiles:
  Data:                single             8.00MiB
  Metadata:            DUP                 51.19MiB
  System:              DUP                 8.00MiB
SSD detected:         no
Incompat features:    extref, skinny-metadata
Number of devices:    1
Devices:
  ID     SIZE  PATH
  1     1.00GiB /dev/sdc
```

Пройдемся по представленным параметрам.

- **Label** — метка, или имя файловой системы. Задается ключом `-L` и является необязательным параметром.
- **UUID** — уникальный идентификатор, благодаря которому ядро `btrfs` отличает инстансы друг от друга.
- **Node size** — размер элементов В-дерева, в которых хранятся метаданные. Его можно задать при помощи ключа `-n | --nodesize`, при этом он должен быть кратен размеру **Sector size**. Малый размер ноды приводит к увеличению высоты В-дерева (увеличению количества нод) и, как следствие, уменьшению конкуренции за блокировку отдельно взятой ноды. С другой стороны, малый размер ноды делает инстанс файловой системы более подверженным фрагментации. Большие же ноды, наоборот, способствуют лучшей упаковке метаданных на диске, что снижает фрагментацию. Обратная же сторона — увеличенное время доступа к данным для обновления одной и той же ноды несколькими потоками. На ядрах старше 3.11 по умолчанию размер ноды равен 16384 байт либо размеру страницы памяти ОС (бОльшему из этих двух значений).
- **Sector size** — объем пространства, кратно которому выделяется и освобождается пространство на физическом уровне. Равен размеру страницы виртуальной памяти ОС, если не указано иное при помощи ключа `-s`.
- **Filesystem size** — суммарная вместимость файловой системы (данные плюс метаданные). Вручную задается ключом `-b`. По умолчанию занимает весь объем блочного устройства.
- **Incompat features** — список возможностей, включенных на созданной `btrfs`, ломающих обратную совместимость со старыми версиями ядер. Если же обратная совместимость необходима, то можно отключить:

```
--features ^extref,^skinny-metadata.Объяснить с
```

Кстати, проверить, какие возможности поддерживает текущее ядро, можно следующим вызовом:

```
mkfs.btrfs --features list-allОбъяснить с
```

- **Number of devices и Devices** — сколько блочных устройств задействовано в созданном инстансе btrfs, и список всех устройств соответственно.
- Отдельно стоит поговорить о параметре **Block Group Profiles**. Он указывает на применяемый профиль записи для каждого из трех типов данных: Data, Metadata и System. Возвращаясь к обобщенной структуре btrfs, можно сказать что:
 - Data — это пользовательские данные;
 - Metadata — это объединение слоя сабволюмов и слоя метаданных и экстентов;
 - System — это структуры отображения адресного пространства физической памяти в непрерывное пространство логических адресов.

Под профилем записи понимается способ хранения данных на физическом уровне:

- Single — хранение данных в единственном экземпляре;
- DUP — дублирование данных на одном носителе;
- RAIDX — одна из конфигураций RAID0, RAID1, RAID10, RAID5 и RAID6.

При разметке одного блочного устройства по умолчанию btrfs применит дублирование к метаданным и системным данным, а пользовательские данные будут оставаться на носителе в единственном экземпляре. Создание же btrfs на нескольких дисках сразу по умолчанию применит профиль «RAID0» к пользовательским данным, а к метаданным — «RAID1».

Управляется данная группа параметров при помощи двух ключей: -d для данных и -m для метаданных и системных данных.

Но есть нюанс... Немного иначе обстоят дела с твердотельными накопителями. Дело в том, что если бы мы размечали SSD диск (или флешку), то по умолчанию файловая система не стала бы дублировать метаданные. Твердотельные накопители для продления времени жизни элементов памяти могут производить дедупликацию данных. Т.е. имея две логические копии данных, по факту на носителе будет записана только одна. Как результат, при выходе сегмента памяти из строя повредятся «обе копии» данных. К тому же, записывая данные дважды, попросту быстрее расходуется ресурс SSD.

Для определения типа носителя btrfs проверяет контент файла `/sys/block/DEV/queue/rotational`, где «DEV» — имя проверяемого блочного устройства.

Разумеется, даже в случае SSD профиль хранения данных можно задать принудительно.

Чтобы создать инстанс btrfs на нескольких устройствах, достаточно указать их через пробел:

```
sudo mkfs.btrfs /dev/sdc /dev/sdd -L double_drive
```

или же с указанием профилей:

```
sudo mkfs.btrfs /dev/sdc /dev/sdd -d raid1 -m raid1 -L raid1_drive
```

Тут надо отметить, что носители не обязаны быть одинакового размера, даже если используется полное зеркалирование. Однако, как только на наименьшем из дисков не хватит места для выделения памяти — файловая система выдаст сообщение об отсутствии свободного пространства, хотя физически на другом носителе некоторое свободное пространство еще может оставаться.

Монтирование

Первое монтирование свежесозданной btrfs не отличается от других файловых систем:

```
mount /dev/sdc /mnt
```

В случае, если файловая система располагается на нескольких дисках, то для монтирования достаточно указать любой из них.

Вообще монтирование btrfs всегда подразумевает монтирование одного или нескольких её сабвоリュームов. Если в вызове команды mount не указывать, какой сабвоリューム необходимо замонтировать, то btrfs прочитает из специальной записи ID сабвоリュームа, который необходимо монтировать по умолчанию. Эту запись в дальнейшем можно изменить командой btrfs set-default, но при первом монтировании на btrfs присутствует только один сабвоリューム — корневой. Именно он указан по умолчанию для монтирования.

Корневой сабвоリューム на btrfs присутствует всегда. Появляется он вместе с файловой системой и в дальнейшем не подлежит каким-либо изменениям.

Для монтирования любого другого сабвоリュームа, кроме дефолтного, существуют два способа:

указать путь от корневого сабвоリュームа btrfs:

```
mount -o subvol=/path/to/subvol /dev/sdc /mnt0
```

Объяснить с

либо указать ID сабвоリュームа:

```
mount -o subvolid=257 /dev/sdc /mnt0
```

Объяснить с

Как уже упоминалось, один из сабвоリュームов btrfs указан как монтируемый по умолчанию. Узнать, какой именно, можно, выполнив:

```
btrfs subvolume get-default /path/to/any/subvolume
```

Объяснить с

Установить сабвоリューム, монтируемый по умолчанию, можно командой:

```
btrfs subvolume set-default 258 /path/to/any/subvolume
```

Объяснить с

Путь до сабвольюма в данном случае нужен только для указания конкретного инстанса btrfs, к которому применяется команда. Кстати, это не обязательно должен быть сабвольюм, подойдет и путь до любой директории.

Команда mount принимает огромное количество опций для управления возможностями btrfs: дефрагментация, сброс кеша, сжатие, cow, логирование, баланс, поддержка ssd и еще вагон разных специфичных для btrfs вещей. Я не буду их рассматривать в рамках данной статьи, т.к. они нужны для тонкой настройки файловой системы, и в подавляющем большинстве случаев можно обойтись и без них.

Subvolume is

Сабвольюм — это ключевой элемент btrfs, исполняющий различные функции:

- хранение в себе пользовательских данных и других сабвольюмов,
- обеспечение доступа к данным (монтирование),
- механизм CoW,
- создание снапшотов.

При первом приближении сабвольюмы — это обычные директории. Их можно переименовывать/перемещать, просматривать их содержимое, размещать и изменять внутри них файлы. Каких-либо специальных утилит при этом не требуется.

Создание и удаление сабвольюма производится на подмонтированной btrfs при помощи специальных команд:

```
btrfs subvolume create /mnt/subvolume_name
btrfs subvolume delete /mnt/subvolume_name
```

Объяснить с

Замечу, что если попытаться удалить сабвольюм средствами файлового менеджера или утилиты **rm**, то операция завершится с ошибкой *operation not permitted* (операция не разрешена).

UPD: Начиная с версии ядра 4.18.0 удаление сабвольюмов можно производить утилитой **rm** или средствами файлового менеджера. Видимо, это был баг, а не фича. Спасибо хабравчанину [Prototik](#) за уточнение.

После создания сабвольюма можно посмотреть его свойства:

```
btrfs subvolume show /mnt/subvolume_name
```

```
Name:                subx
UUID:                09af45e8-d2b2-b342-8a92-fa270ac82d0a
Parent UUID:         -
Received UUID:       -
Creation time:       2019-03-23 17:59:28 +0100
Subvolume ID:        268
Generation:          39
Gen at creation:     35
Parent ID:           260
Top level ID:        260
Flags:               -
Snapshot(s): Объяснить с
```

Пройдемся по основным свойствам сабвольюма:

- **Name** — имя сабвольюма,
- **UUID** — универсальный уникальный идентификатор, служащий, в основном, для определения связей сабвольюм-снэпшот,
- **Parent UUID** — идентификатор сабвольюма-предка, от которого произведен текущий,
- **Received UUID** — идентификатор сабвольюма-предка, отправленного через *btrfs send*,
- **Subvolume ID** — уникальный идентификатор для размещения в B-дереве,

- **Generation** — номер транзакций при последнем обновлении метаданных сабвольюма,
- **Gen at creation** — номер транзакции на момент создания сабвольюма,
- **Parent ID** — идентификатор сабвольюма, в который вложен текущий,
- **Top level ID** — абсолютно то же самое, что и Parent ID,
- **Flags** — флаги (по факту только 1 флаг — *readonly*),
- **Snapshots** — список снапшотов, снятых с данного сабвольюма.

У сабвольюма есть еще один параметр — это его путь от корневого элемента btrfs. Путь отображается при выводе списка сабвольюмов:

```
btrfs subvolume list /path/to/any/btrfs/mountpoint
```

Объяснить с

Но тут все просто и наглядно — даже нет смысла приводить вывод команды. Так же, как и с командами `get-default` и `set-default`, тут можно указать путь до любого сабвольюма, результат от этого не изменится. Этот путь используется для того, чтобы найти корневой сабвольюм btrfs. После чего происходит чтение всего дерева сабвольюмов.

Если попытаться скопировать сабвольюм, например, утилитой **cp**, то операция копирования выполнится успешно, но в результате будет создан не сабвольюм, а обычная директория. Однако btrfs предоставляет гораздо более гибкий инструмент для создания подобных копий — снапшоты.

Snapshot is

Снапшот — это тоже сабвольюм, просто обладающий расширенными свойствами.

Основное их отличие — наличие у снапшота записей о том, от какого сабвольюма он был произведен. Это поля **Parent UUID** и **Received UUID**. У сабвольюма эти поля тоже присутствуют, но они всегда пустые. Так что, по сути, снапшот и сабвольюм — это одно и то же.

Снапшот при создании можно заблокировать для изменений при помощи ключа `-r`.

```
btrfs subvolume snapshot -r /path/to/subvol /path/to/snapshot
```

В этом случае файлы гарантировано останутся в том состоянии, в котором они пребывали на момент создания снапшота.

Флагом «только для чтения» так же можно управлять вручную, это работает для любого сабвольюма:

```
btrfs property get /path/to/subvol ro
btrfs property set /path/to/subvol ro true
```

Если теперь заглянуть в свойства снапшота, то увидим заполненное поле **Parent UUID**:

```
btrfs subvolume show /path/to/snapshot
```

```
Name:                subx
UUID:                d08612d8-596a-11e9-8647-d663bd873d93
Parent UUID:        09af45e8-d2b2-b342-8a92-fa270ac82d0a
Received UUID:      -
Creation time:       2019-03-23 17:59:28 +0100
Subvolume ID:       269
Generation:         39
Gen at creation:    35
Parent ID:          260
Top level ID:       260
Flags:              -
Snapshot(s):
```

Важной особенностью операции создания снапшота является то, что она нерекурсивна. Вместо вложенных сабвольюмов в снапшоте будут созданы пустые директории.

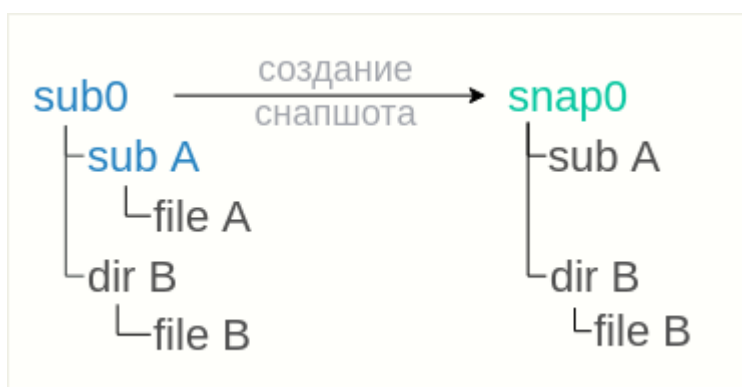
Обратимся к следующему примеру.

На файловой системе имеется сабвоリューム «sub0», внутри которого расположен сабвоリューム *subA* и директория *dirB*. Внутри каждого из них расположены *fileA* и *fileB* соответственно.

Снимаем снимок:

```
btrfs subvolume snapshot sub0 snap0
```

объяснить с



Созданный снимок *snap0* унаследует все файлы и директории своего родителя, однако, сабвоリューム *subA* внутри снимота не появится. Вместо него в снимоте появится только пустая директория, т.е. содержимое сабвоリュームа *subA* унаследовано не будет.

С одной стороны, это хорошо — мы снимаем снимок с конкретного сабвоリュームа, и все вложенные нас не интересуют. С другой стороны — в случае, если требуется рекурсивный снимок, то у *btrfs* нет решения данной задачи. Придется искать воркэраунды.

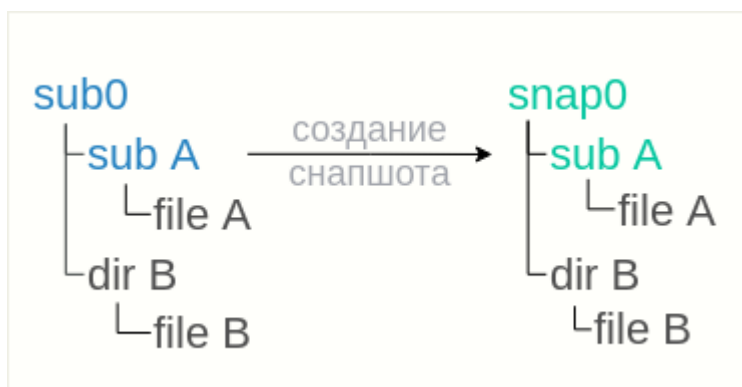
Первый обходной путь основывается на том, что снимок был снят без флага «только для чтения», что позволяет исправить ситуацию достаточно просто:

- удалить лишнюю директорию из снимка

```
rm -rf snap0/subA
```

- снять снимок с вложенного сабволюма

```
btrfs subvolume snapshot sub0/subA snap0/subA
```

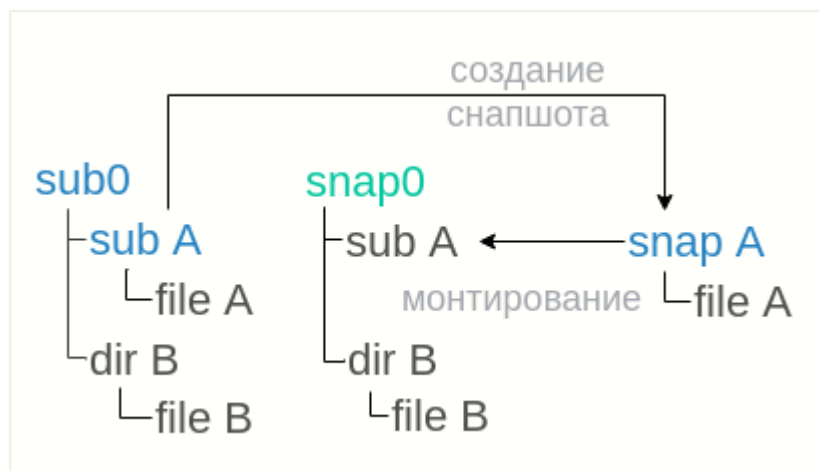


Если же снимок был снят с флагом «только для чтения», то приведенный вариант не сработает, т.к. в *snap0* нельзя ни удалить директорию, ни разместить снимок. Тут вариант только один — размещать снимки где-то рядом с сабволюмом *snap0*:

```
btrfs subvolume snapshot sub0/subA snapA
```

а затем монтировать *snapA* внутри снимка *snap0*, директория для этого уже имеется:

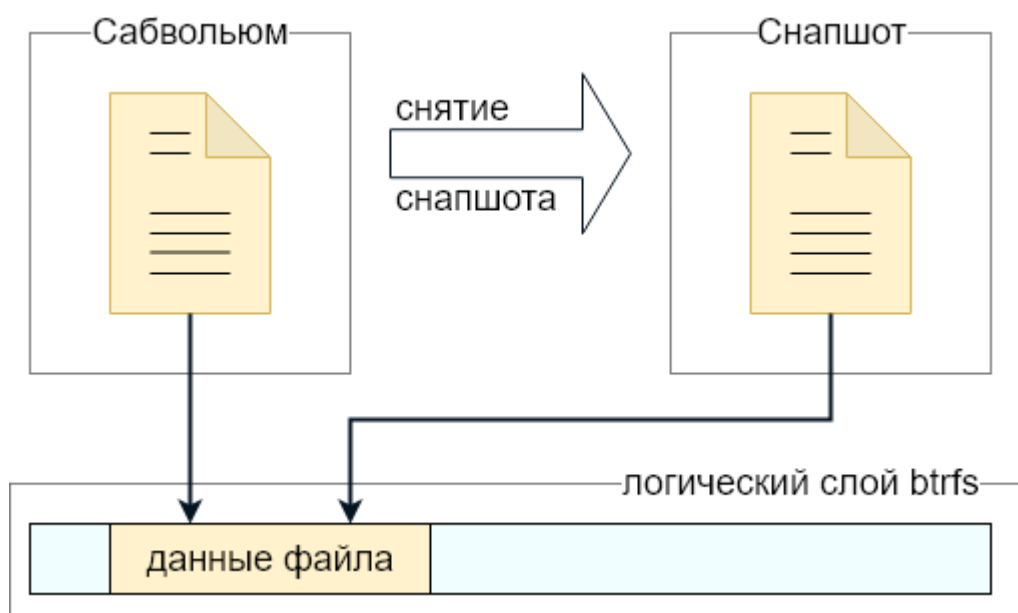
```
mount -o subvol=snapA snap0/subA
```



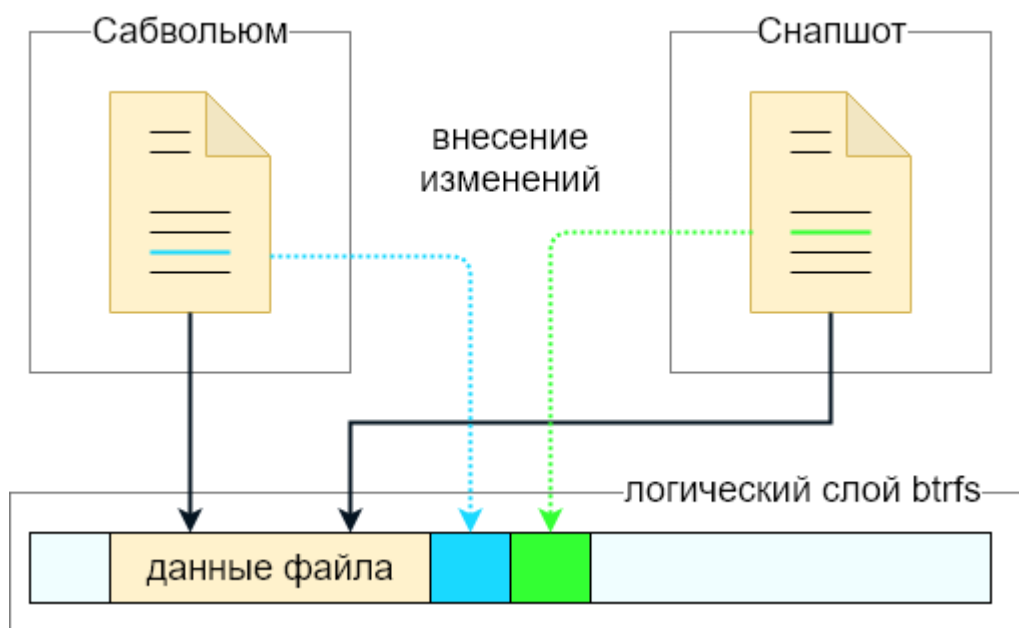
В любом случае важно понимать, что рекурсивные снимоты все будут сняты в рамках разных операций, в разное время. Ни о каком атомарном снятии снимота с нескольких сабволюмов речи быть не может.

Copy on write

Немного о сабволюмах и CoW-подходе. Представим, что на файловой системе присутствует сабволюм и в нем расположен файл (возьмем идеальный случай — файл не фрагментирован). Далее с сабволюма снимается снимот.



На файловой системе появится новый сабвоリューム (снимок) ровно с тем же самым содержимым, которое было у исходного сабвоリュームа. Процесс создания снимка проходит почти мгновенно — данные самого файла не копируются. Вместо этого создаются дополнительные метаданные, и снимок наравне с родительским сабвоリュームом становится владельцем файла. По сути, файл на диске остался один, но теперь принадлежит одновременно как сабвоリュームу, так и снимку. Если теперь изменить файл в сабвоリュームе, то изменения никак не повлияют на файл в снимке. Если при создании снимку не был выставлен флаг «только для чтения», то файл в снимке также можно изменять.



Технически, при изменении файла записываются только эти изменения. Так что на диске будет храниться исходный файл плюс некоторая дельта, отличающая исходный файл от измененного. Если же удалить один из сабвоリュームов (под вторым я имею в виду снимок), то лишние данные, более никем не используемые, будут вычищены с диска, а на диске останется только актуальная версия файла (с точки зрения оставшегося сабвоリュームа).

Небольшое замечание: При удалении сабвоリューム исчезнет с глаз пользователя мгновенно, и утилита вернет управление в терминал, однако сами данные на диске будут вычищены фоновым процессом в течение некоторого времени. Т.е., в отличие от удаления обычной директории, здесь не придется ждать реального завершения операции удаления. Если же необходимо синхронизироваться с этим процессом и дождаться его завершения, можно при вызове **delete** указать ключ `--commit-after`. Команда **btrfs subvolume list**, вызванная с ключом `-d`, выведет список сабвоリュームов,

которые были удалены пользователем и на текущий момент находятся в процессе удаления с диска.

Ко всему прочему, btrfs позволяет клонировать файлы на файловой системе, не прибегая к снапшотам. Делается это обычным копированием с указанием ключа --reflink:

```
cp -ax --reflink=always /original/file /copied/file
```

Объяснить с

Ключ reflink=always сообщает файловой системе, что мы хотим задействовать механизм CoW при копировании. После копирования файлы можно изменять независимо друг от друга, так что мы получаем то же самое поведение, как и после создания снапшота. Так зачем тогда нужны сабвольюмы?

Сабвольюмы на btrfs играют роль высокоуровневого средства управления целыми наборами данных: во-первых, это атомарное снятие снапшота со всех данных сабвольюма (в случае с --reflink атомарность лишь на уровне файла), во-вторых — есть возможность посмотреть, кто от кого наследуется, или быстро «откатить» набор данных на более раннюю версию, и т.п.

Таким образом, btrfs предоставляет возможность фиксировать состояния файлов в желаемые моменты времени, применяя сабвольюмы как высокоуровневое средство управления этими состояниями.

Восстановление сабвольюма

На просторах необъятного часто встречается вопрос: «У меня есть сабвольюм, у меня есть снапшот, как сделать реверт?» Данный подход не применим к btrfs, т.к. нет самой возможности «откатить сабвольюм». Вместо этого btrfs предлагает стратегию замены сабвольюма на его снапшот. Действительно, зачем что-то ревертить, если сам снапшот — это и есть тот объект, который мы хотим получить при помощи реверта.

Представим себе такой сценарий: на btrfs расположен сабвоリューム, в котором располагаются файлы какой-либо базы данных (ну или другие важные данные). С этого сабвоリュームа периодически снимаются снапшоты, и в определенный момент возникает необходимость откатить данные. В этом случае мы просто избавляемся от сабвоリュームа и вместо него начинаем использовать снятый с него снапшот, либо — если не хотим испортить еще и эти данные — снимаем со снапшота еще один снапшот. Если оригинальный сабвоリューム не был замонтирован и использовался как обычная директория, то его необходимо либо удалить либо переместить/переименовать, а на его место поместить снапшот.



В консоли это может выглядеть примерно так:

- переименовываем сабвоリューム

```
mv the_subvolume the_subvol.old
```

Объяснить с

- помещаем на место сабвоリュームа его снапшот

```
btrfs subvolume snapshot the_snapshot the_subvolume
```

Объяснить с

Если же сабвоリューム был замонтирован и использовался через точку монтирования, то достаточно отмонтировать сабвоリューム и подмонтировать на его место снапшот.



- Отмонтируем сабвоリューム

```
umount /mnt/Объяснить с
```

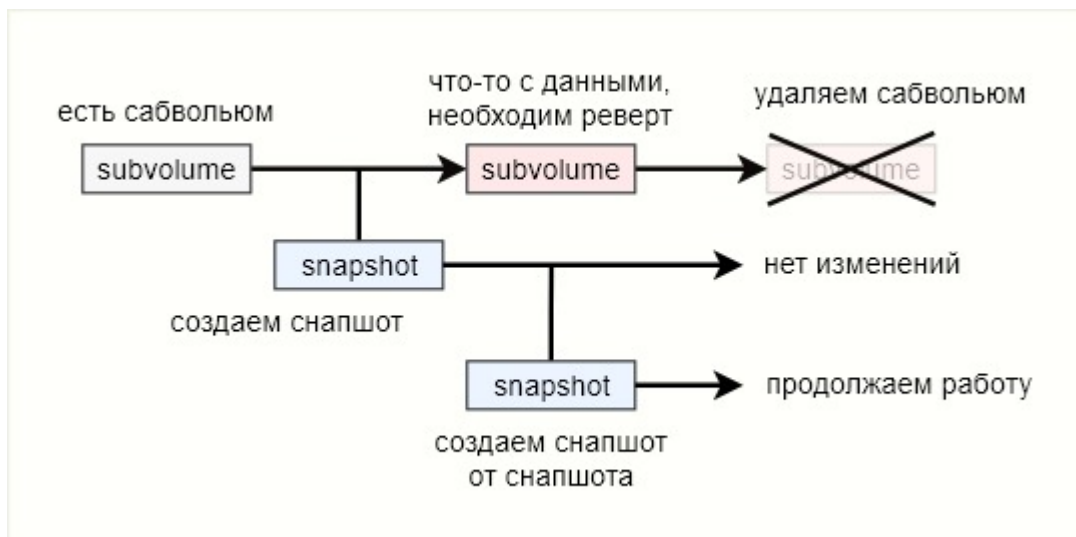
- Можно создать снапшот снапшота, чтобы не испортить последние уцелевшие данные:

```
btrfs subvolume snapshot /path/to/snapshot /path/to/snapshot_copyОбъяснить с
```

- монтируем снапшот:

```
mount -o subvol=path/to/snapshot_copy /mntОбъяснить с
```

Для полноты картины попробую еще раз и немного по-другому. Сабвоリューム, в котором происходят изменения — это ветка *main*.

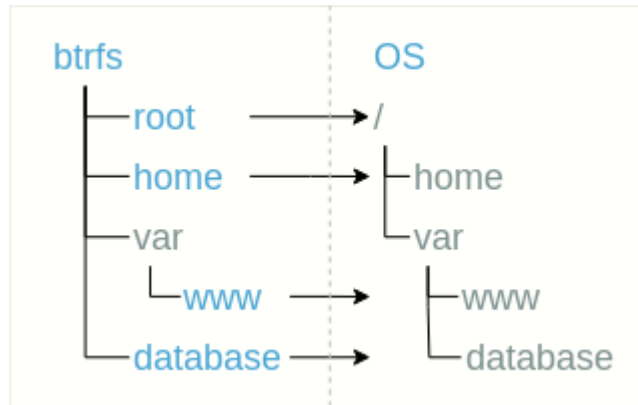


При создании снапшота происходит фиксация состояния файлов на диске. С этого момента снапшот — это бранч ветки *main*. Все дальнейшие изменения в *main* никак не повлияют на снапшот. Откат же к снапшоту означает прекращение использования ветки *main* и полное переключение на бранч. Ветку *main* при этом за ненадобностью можно удалить. Таким образом, btrfs — это практически система контроля версий, но без возможности обратно сmerzить ветки.

Дерево файловой системы

Одним из неочевидных моментов, связанных с использованием btrfs, является то, как следует разбивать данные системы на сабвolumы. Разумеется, какого-либо «правильного» подхода в данном вопросе не существует. Но можно выделить 3 способа организации структуры сабвolumов: плоская структура, вложенная и смешанная.

Плоская структура означает расположение сабвolumов плоским списком в корневом сабвolumе. Например, отдельными сабвolumами можно выделить корень файловой системы (назовем его *root*), пользовательскую директорию *home*, директорию с сайтом */var/www* и базу данных, расположенную например в */var/database*.



Некоторые сабволюмы для удобства можно размещать в директориях, как, например, в случае с сабволюмом *var/www*.

При таком подходе все сабволюмы необходимо замонтировать. Сабволюм *root* должен иметь точку монтирования */*, а внутри себя содержать директории *home* и *var*. После монтирования *root* в */home* должен монтироваться сабволюм *home*, а в */var/www* и */var/database* — сабволюмы *var/www* и *database* соответственно.

Таким образом, дерево btrfs-сабволюмов можно произвольным образом отобразить в виртуальной файловой системе ОС, а там уже на что фантазии хватит.

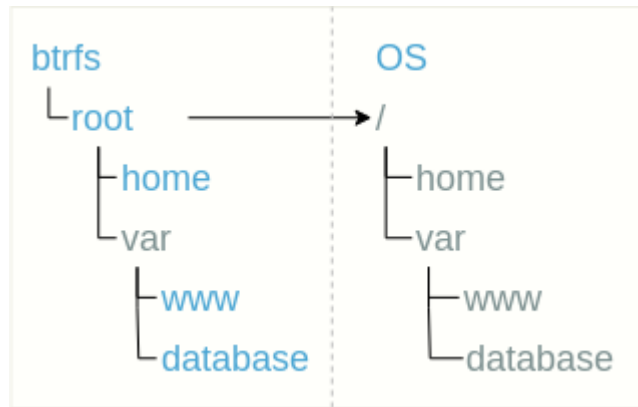
Плюсы:

- пользователю видны только замонтированные сабволюмы,
- легко заменить сабволюм (отмонтировать один, примонтировать другой),
- легко удалить сабволюм.

Минусы:

- легко запутаться что куда монтируется,
- для каждого сабволюма должна быть запись в *fstab*, а если происходят «откаты» к снапшотам, то соответствующие записи в *fstab* необходимо еще и обновлять.

Вложенная структура сабволюмов предполагает простое использование сабволюмов вместо некоторых директорий.



В этом случае кроме корневого сабвоリュームа монтировать больше ничего не требуется.

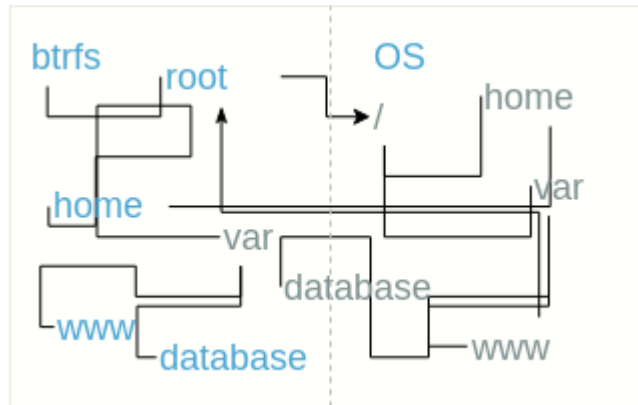
Плюсы:

- все сабвоリュームы видны, структура проста для восприятия,
- не нужно ничего монтировать лишний раз, все как с «обычной» файловой системой.

Минусы:

- все сабвоリュームы видны, возможно, некоторые хотелось бы спрятать от пользователя,
- сложно удалить/заменить сабвоリューム (причиной тому — вложенные сабвоリュームы).

Ну и третий подход — смешанный. Он предполагает комбинацию первых двух для извлечения максимальной выгоды из обоих. Однако, не исключено что именно данный подход приведет к сложной, тяжело изменяемой, запутанной структуре с огромным количеством записей в `fstab`. Все зависит от трезвости мастера сисадмина.



Добавление/удаление диска, баланс

btrfs может похвастаться прекрасным функционалом — возможностью «на горячую» добавить блочные устройства непосредственно в процессе работы файловой системы:

```
btrfs device add /path/to/device /path/to/btrfs
```

Объяснить с

Или удалить:

```
btrfs device remove /path/to/device /path/to/btrfs
```

Объяснить с

Кстати, в одном вызове добавления/удаления можно указать несколько дисков. Опять же, указываемый путь — это путь до любого сабвolumes той btrfs, к которой будет применена команда.

Проверим, сколько и какие блочные устройства находятся под управлением btrfs:

```
btrfs filesystem show /path/to/btrfs
```

```
Label: none  uuid: 52961dda-df84-4e2d-9727-e93e7738df81
```

```
Total devices 2 FS bytes used 192.00KiB
devid    1 size 20.00GiB used 132.00MiB path /dev/sdc
devid    2 size 50.00GiB used 0.00B path /dev/sdd
```

Объяснить с

0.00B в поле **used** нам сообщает о том, что добавленный диск пуст. Для наполнения его данными согласно профилю записи необходимо выполнить баланс:

```
btrfs balance start /path/to/btrfsОбъяснить с
```

Команда **balance** перераспределяет данные на дисках согласно выбранному профилю записи. Например, в случае RAID1 баланс приведет к клонированию данных с первоначального устройства, в случае RAID0 — к более равномерному распределению данных по двум дискам, и т.д.

В результате баланса, если до этого на диске присутствовали пустоты, то данные на диске будут записаны более плотным образом, т.е. получится дефрагментация.

Однако важно понимать, что это не совсем «та» дефрагментация. В данном случае команда **balance** не смотрит на логическое содержимое, а оперирует лишь блоками данных. Она не обращает внимание на то, что какой-либо файл размазан по диску. Вместо этого **balance** переносит блоки данных из одного места в другое. Т.е. файл, фрагментированный до баланса, останется фрагментированным и после него. Но! Фрагментированность на уровне блоков данных все же уменьшится, и этим можно пользоваться.

Для избежания путаницы скажем так: операция **balance** уменьшает фрагментированность на уровне блоков данных, но не влияет на фрагментированность файлов.

Также команда **balance** предоставляет возможность сменить профиль записи. Например, на диске использовался профиль DUP, а после добавления диска решили сделать полноценный RAID1. Для этого необходимо воспользоваться фильтром `convert`:

```
btrfs balance start -dconvert=raid1 -mconvert=raid1 /path/to/btrfs0
```

Объяснить с

При помощи опций `-dconvert` и `-mconvert` задаются новые профили записи для данных и метаданных соответственно. Существует также опция `-sconvert`, которая предназначена для смены профиля записи системных данных, однако с ней придется еще дописать ключик `-f` (`--force`) для принудительного выполнения операции.

Вообще, основное назначение фильтров — это задание правил для операции `balance`: какие блоки обработать, а какие не трогать. Так, например, можно затронуть только блоки, записываемые с определенным профилем записи (фильтр `profiles`), или блоки занятые выше определенного процента (фильтр `usage`), либо затронуть только группы блоков, имеющих отношение к определенному диску (фильтр `devid`) и т.п. Кстати, их еще можно комбинировать. В целом возможности фильтров очень обширны и в основном необходимы для проведения выборочного баланса данных.

Фрагментация

К сожалению, `btrfs` «благодаря» своей архитектуре крайне подвержена такому явлению, как фрагментация. Дело в том, что данные записываются всегда в новое расположение на диске. Даже если прочитать файл, ничего не сделать с данными и записать их обратно в тот же файл, то данные попадут на диске в новую область. То же самое произойдет, если обновить данные в файле только частично — изменения запишутся в новую область на диске. Таким образом, частые изменения весьма сильно фрагментируют файлы, увеличивая «разбросанность» фрагментов, в общем случае — по нескольким дискам. Это приводит к увеличенной нагрузке на CPU и неоправданному расходу памяти. Сильнее всего фрагментированности подвержены базы данных и образы виртуальных машин.

Оценить фрагментированность файлов можно при помощи утилиты **filefrag** (не входит в `btrfs-progs`).

```
filefrag /path/to/your/file0
```

Объяснить с

Она показывает количество экстентов, задействованных для хранения файла. Проще говоря — чем меньше экстентов задействовано, тем меньше фрагментированность файла.

Для борьбы с фрагментацией на btrfs существует два метода: дефрагментация и флаг `posow`.

Дефрагментацию можно применять к отдельному файлу либо к сабвольту/директории, в т. ч. рекурсивно. Команда выглядит следующим образом:

```
btrfs filesystem defragment /path/to/file/or/dir
```

Объяснить с

Надо сказать, что не всегда эта команда приводит к ожидаемым результатам. Небольшие несильно фрагментированные файлы (10 — 20 экстентов) после дефрагментации могут оказаться разбиты на еще большее число частей. К тому же на некоторых версиях ядра дефрагментация btrfs приводит к поломке дедуплицированности файлов, производя их реальные физические копии. Т.е. снапшоты на физическом уровне станут полноценными копиями.

Второй способ борьбы с фрагментированностью — это атрибут файла `posow`.

```
chattr +C /path/to/file
```

Объяснить с

Атрибут `posow` можно выставить только новому или пустому файлу. Он отключает механизм *copy on write*, благодаря чему btrfs при обновлении содержимого файла будет всегда работать с фиксированной дисковой областью, записывая данные поверх существующих (на физическом уровне). Из минусов `posow` — он отключает еще и проверку чексуммы для данного файла. Другими словами, нет `cow` — нету и `checksum`.

Разумеется, вручную выставлять атрибут `posow` каждому файлу — это дело неблагодарное. Если выставить этот флаг директории/сабвольтюму, то все новые файлы, созданные в нем, унаследуют флаг автоматически. Это же касается и создаваемых вложенных директорий. Если же на момент включения атрибута в директории уже находились какие-либо данные, то на них это никак не повлияет — атрибут `posow` можно выставить только новому или пустому файлу.

И еще один способ автоматического выставления флага `posow` — это монтирование файловой системы с указанием опции `nodatacow`:

```
mount -o subvol=path/to/subvol,nodatacow /dev/sdXX /path/to/mountpoint
```

Объяснить с

Данная опция вызовет автоматическое подключение опции `nodatasum`, так что для новых создаваемых файлов чексуммы вычисляться не будут.

Как обычно, есть нюанс: нельзя подмонтировать только один сабвольтюм с опцией `posow`. Либо все сабвольтюмы будут иметь опцию `posow`, либо ни один. Все решает первый подмонтированный сабвольтюм: если у него была указана опция `nodatacow`, то и все последующие монтирования пройдут с этой опцией автоматически.

Неочевидный момент возникает, если выставить файлу флаг `posow` и снять снимок с сабвольтюма, в котором этот файл расположен. В этом случае `btrfs` игнорирует флаг `posow`, если на обновляемый блок данных ссылается более одного сабвольтюма. Поэтому, несмотря на флаг `posow` (кстати, файл в снимке его также унаследует), изменения любого из файлов будут попадать на диск в новую область, и файл снова станет фрагментированным. Если же блок данных в файле обновить несколько раз, то первый раз он попадет в новую область на диске, а при последующих записях будет обновляться в этой новой области «на месте».

Трюки и фейлы

При использовании `btrfs-progs` можно не писать полное название команды:

```
btrfs sub cre = btrfs subvolume create Объяснить с
```

Достаточно лишь совпадения первых символов, которые однозначно определяют команду:

```
su = subvolume,  
fi = filesystem,  
ba = balance,  
de = device;  
Объяснить с
```

думаю, принцип понятен.

Создать снапшот директории, увы, btrfs не под силу, но есть обходной путь:

- создайте сабвolumes

```
btrfs subvolume create ./subvol0 Объяснить с
```

- скопируйте файлы из директории в сабвolumes:

```
cp -ax --reflink=always your/dir/. ./subvol0 Объяснить с
```

с ключом `reflink=always` будет задействован механизм CoW, т.е. данные не будут скопированы, но будут созданы ссылки на них на низком уровне btrfs.

- Теперь сабвоリューム содержит все файлы, которые были в директории, и можно снять с него снапшот.

Выставить атрибут `posow` существующему файлу с данными нельзя. Однако можно пойти следующим путем:

- создать пустой файл

```
touch poscowfile0
```

- выставить ему флаг `posow`
- выделить место на диске под новый файл

```
fallocate -l10g poscowfile0
```

- скопировать содержимое существующего файла в

```
cp -v oldcowfile poscowfile0
```

Если на `btrfs` закончилось место, то даже удаление какого-либо файла может вызывать ошибку «*No space left on device*». Для решения рекомендуется подключить к `btrfs` временный накопитель размерами желательно не менее 1GB. После чего произвести чистку данных. Затем удалить временный накопитель.

Операция **balance**, вызванная без указания профилей записи, неявно меняет их с *dup* на *raid1*. О чем, кстати, написано на странице [Gotchas](#). Происходит это после добавления диска к `btrfs`, на которой используется профиль записи *dup*. Напомню, что форматирование одиночного диска в `btrfs` использует профиль *dup* по умолчанию для метаданных и системных данных.

Пожалуй, самое важное

Избегайте создания низкоуровневых клонов блочных устройств с btrfs. Будучи «умной» файловой системой, при некоторых операциях (чаще всего, при монтировании) btrfs самостоятельно перечитывает системные данные на блочных устройствах, чтобы найти все части файловой системы. Если в процессе поиска будет обнаружено два блочных устройства с одинаковыми UUID, то btrfs воспримет их как части одного и того же инстанса. Если же при этом эти два устройства окажутся оригиналом и его клоном, то после монтирования одному только драйверу известно, как будет происходить работа файловой системы, но ясно, что ничем хорошим это не закончится. В худшем случае — закончится необратимым повреждением данных.

Если же очень хочется клонировать диски с btrfs низкоуровневым способом, то необходимо соблюдать крайнюю осторожность. В общем случае клон не должен быть виден ядру ОС как блочное устройство, пока в системе присутствует оригинал, и наоборот. Обеспечив это условие, можно изменить UUID клона (ну или оригинала, тут по желанию). В этом поможет утилита **btrfstune**, которая поставляется вместе с пакетом **btrfs-progs**:

```
btrfstune -u /path/to/device
```

Объяснить с

И снова: **btrfstune**, будучи «умной» утилитой, будет изменять UUID не только на диске, а на всей файловой системе. Это значит, что при вызове она пойдет читать все блочные устройства, дабы заменить UUID на всех устройствах, относящихся к файловой системе.

Вместо заключения

Если на данном моменте вы ничего не поняли — это нормально. Btrfs нетривиальна и сразу может не поддаться. Каждый раз, когда мне казалось, что вот теперь-то я её понял, она подкидывала сюрприз и заставляла переосмысливать существующие

вещи. Не могу сказать, что я все понял и на текущий момент — в процессе написания находил что-то новое, хотя и писал уже на основе имеющегося опыта.

Я бы сравнил процесс освоения `btfs` с переходом от процедурного стиля программирования к объектно-ориентированному. Первое впечатление — «вау как круто», но затем упорно продолжаешь писать процедурный код, обернутый в классы.

В статье я старался не лить воду — писать все по делу. Не смотря на это, получилось довольно объемно. Но рассказать удалось далеко не все — про `btfs` можно еще писать и писать. Эта статья — лишь верхушка айсберга. Самое начало, чтобы понять её философию и начать использовать. А сейчас на этом пора заканчивать.

Спасибо, что дочитали до конца. Надеюсь, не утомил. Пишите в комментариях, о чем Вам еще бы было интересно узнать.

Делайте бэкапы, господа. И пусть они вам никогда не пригодятся.